

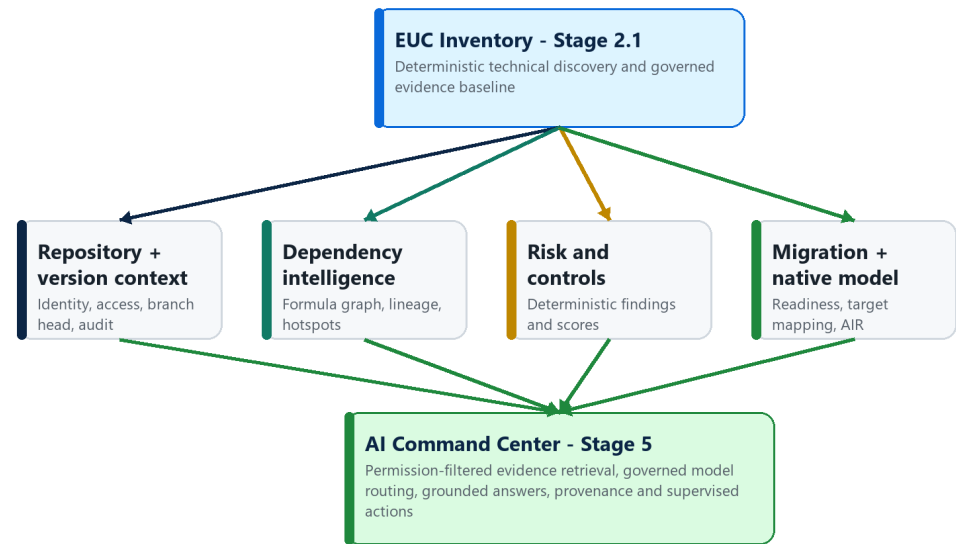
TECHNICAL ARCHITECTURE

EUC Inventory and Grounded AI Integration

Why it was built, what was implemented, how the code works, and how verified EUC evidence should connect to AI

EUC Inventory in the Git Walk platform

High-level feature relationship and evidence flow



Git Walk technical feature documentation | Current implementation and controlled target state

Document type	Technical architecture and implementation guide
Scope	EUC Inventory Stage 2.1 and its direct AI integration
Code baseline	Repository branch ver2, reviewed 26-27 August 2026
Status	Code-grounded documentation; target-state items explicitly labeled

Document purpose and reading guide

This document explains the EUC Inventory capability implemented in Git Walk, beginning with the business and engineering rationale and then tracing the feature from the React user interface through FastAPI, validation, immutable storage, static analyzers, normalized persistence, downstream intelligence, and AI grounding.

ACCURACY CONVENTION

Implemented means directly present in the reviewed code. Downstream integration means implemented elsewhere in the repository and dependent on Stage 2.1. Recommended target state identifies a documented extension that is not yet wired end to end.

Contents

- 1. EUC Inventory feature overview
- 2. Scope, terminology and architectural principles
- 3. High-level platform architecture
- 4. Detailed Stage 2.1 architecture
- 5. Feature block specifications
- 6. Data model and persistence
- 7. API and user-interface contracts
- 8. EUC Inventory and AI Command Center
- 9. Security, governance and non-functional behavior
- 10. Testing and acceptance strategy
- 11. Advantages, limitations and recommended improvements
- 12. Codebase traceability and dependencies
- Appendices: operational runbook, glossary and acceptance checklist

1. EUC Inventory feature overview

1.1 Why the feature was built

Enterprise spreadsheets frequently operate as applications: they implement calculations, approvals, forecasts, data transformations, reports and integrations. Conventional file repositories can identify a workbook by name and binary content, but they cannot explain its internal logic, dependencies or operational risk. This creates an inventory problem before it creates an AI problem.

Git Walk therefore introduces EUC Inventory as the technical-discovery entry point for spreadsheet intelligence. The feature establishes a deterministic, versioned and permission-controlled record of what exists inside a workbook. This baseline is necessary before dependency analysis, risk scoring, migration planning or AI explanation can be trusted.

FEATURE DEFINITION

EUC Inventory is a static-analysis pipeline for .xlsx, .xlsm and UTF-8 .csv assets. It fingerprints and preserves the source, extracts supported technical structures without executing workbook code, normalizes the results, and seals an immutable inventory manifest.

1.2 What was built

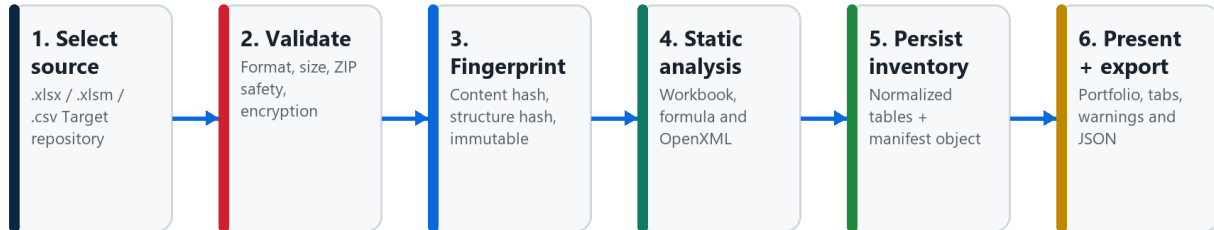
- A repository-scoped EUC registration and portfolio experience in React.
- Authenticated FastAPI endpoints for upload, analysis, status, inventory retrieval and JSON export.
- Hostile-file-aware validation for OpenXML and CSV inputs.
- Content and structure fingerprints for identity, deduplication and provenance.
- Immutable storage of original source bytes in the semantic object ledger.
- Three static analyzers covering workbook structure, formula patterns and OpenXML package metadata.
- Normalized SQLite tables for sheets, formulas, occurrences, objects, external links, connections and warnings.
- A versioned immutable analysis manifest associated with the repository head commit and analyzer version.
- Downstream consumption by dependency, risk/control, migration and application-model features.
- An AI evidence path that currently retrieves deterministic EUC findings and repository evidence for grounded explanation.

1.3 How it was built

The implementation follows a compiler-like pipeline. Input bytes pass through validation and identity gates, are preserved as immutable source evidence, and are loaded into a controlled analysis context. Independent analyzers emit metrics, normalized records and warnings. A service layer persists those records transactionally and creates a canonical manifest. The UI then queries the latest inventory by section.

EUC Inventory end-to-end lifecycle

Implemented Stage 2.1 processing path



Invariant

The original source is not executed or mutated; every result is tied to source identity and analyzer version.

Figure 1. Implemented end-to-end EUC Inventory lifecycle.

1.4 User-visible capability

The EUC Inventory page allows an authorized user to select a target repository, register a workbook or CSV, view analysis status, search the governed portfolio, inspect workbook-map and formula/object details, re-run analysis, and export the current manifest as JSON. The page also exposes downstream tabs; those tabs depend on Stage 2.1 but are separate feature stages.

2. Scope, terminology and architectural principles

2.1 Scope boundary

Included in this document	Excluded except as a dependency
Stage 2.1 upload, validation, fingerprints, source storage, analyzers, inventory persistence, portfolio, export	General repository version control implementation
Direct internal dependencies needed by Stage 2.1	Full dependency-engine algorithms
Current and recommended EUC-to-AI evidence connection	General integrations, digital-thread and merge workflows
Testing and operating the EUC feature	Unrelated frontend and backend features

2.2 Core terms

Term	Technical meaning
EUC asset	A registered .xlsx, .xlsm or .csv source associated with a Git Walk repository.
Content fingerprint	SHA-256 of the exact source bytes; used for identity and repository-scoped deduplication.
Structure fingerprint	SHA-256 of a canonical representation of workbook package structure or CSV header.
Analysis run	A versioned execution of analyzer version 2.1.0 against an immutable source.
Inventory	Normalized queryable records describing sheets, formulas, objects, links, connections and warnings.
Manifest	Immutable JSON evidence object containing the summary and extracted records for an analysis run.
Grounded AI	AI output constrained to authorized evidence references, structured response validation and provenance.

2.3 Architectural principles

1. Deterministic extraction is authoritative. Workbook counts and structures come from parsers, not an LLM.
2. No workbook code is executed during inventory. VBA and connections are inspected as metadata only.
3. Every analysis is tied to exact source identity, repository context, analyzer version and a manifest hash.
4. Authorization is applied before repository data or EUC evidence is read or changed.
5. AI explains verified evidence; it must not invent inventory facts or override deterministic controls.
6. Target-state capabilities are documented separately from current implementation.

3. High-level platform architecture

Stage 2.1 sits between repository governance and higher-order intelligence. Repository context supplies identity, access, branch-head provenance and audit. EUC Inventory supplies the normalized technical baseline. Dependency and intelligence stages derive relationships and findings. Migration and application-model stages consume those facts. Stage 5 AI retrieves authorized evidence and produces explanations with provenance.

EUC Inventory in the Git Walk platform

High-level feature relationship and evidence flow

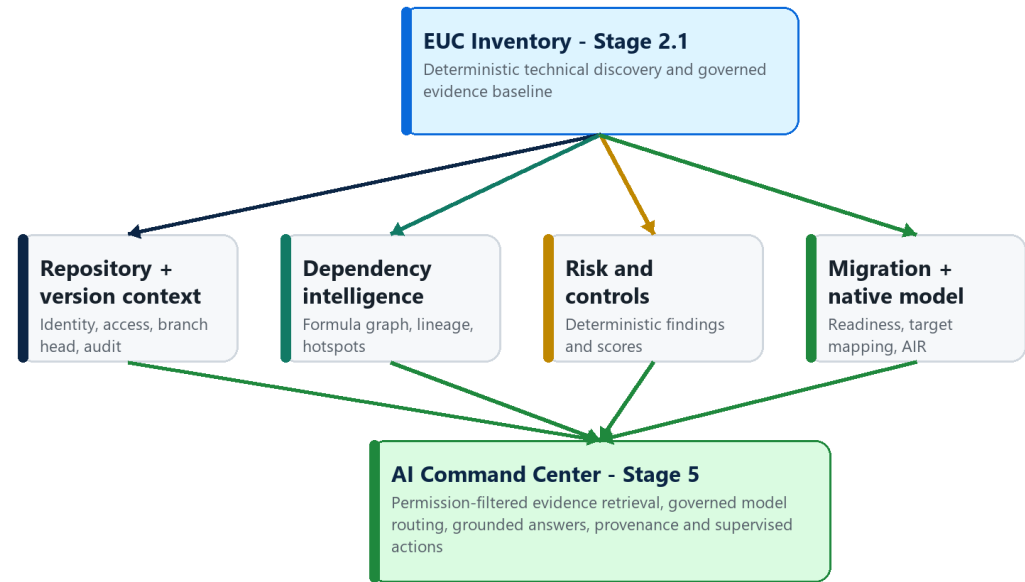


Figure 2. High-level relationship between EUC Inventory and direct downstream platform features.

Connected feature	Connection to EUC Inventory	Status
Repository governance	Repository ID, membership, organization, default-branch head and audit events	Implemented direct dependency
Semantic storage	Immutable original source and immutable analysis manifest	Implemented direct dependency
Dependency intelligence	Consumes formula occurrences, sheet identities, external links and connections	Implemented downstream stage
Risk and controls	Consumes inventory and graph-derived features to generate deterministic findings	Implemented downstream stage
Migration / native model	Uses technical facts, findings and dependencies for readiness and target models	Implemented downstream stage
AI Command Center	Currently retrieves repository evidence and open EUC findings; direct raw inventory adapter is recommended	Partial direct integration

4. Detailed Stage 2.1 architecture

4.1 Runtime components

Layer	Primary components	Responsibility
Presentation	frontend/src/App.jsx; styles.css	Portfolio, upload, selection, inventory tabs, warnings, re-analysis and export.
Client API	frontend/src/services/api.js	Multipart upload and authenticated REST calls under /api/v1/euc.
HTTP boundary	backend/app/api/euc.py	FastAPI routing, request constraints, principal dependency and typed HTTP errors.
Application service	backend/app/euc/service.py	Access checks, deduplication, orchestration, persistence, status, retrieval and export.
Analysis	validation.py; fingerprint.py; analyzers.py	Safety gates, identity and static extraction.
Persistence	database.py; semantic_ledger_service.py	Normalized SQLite records, object storage, references and immutable evidence.
Governance	access_control; security; observability	Permissions, principal, audit events and metrics.

4.2 Synchronous processing sequence

The current frontend performs registration and analysis as two sequential HTTP requests and waits for analysis completion. Although analysis-run status fields exist, POST /analysis executes synchronously in the current API. Large workbooks can therefore hold an HTTP request until completion or the configured timeout.

UI and API interaction

Synchronous Stage 2.1 sequence implemented in the current frontend

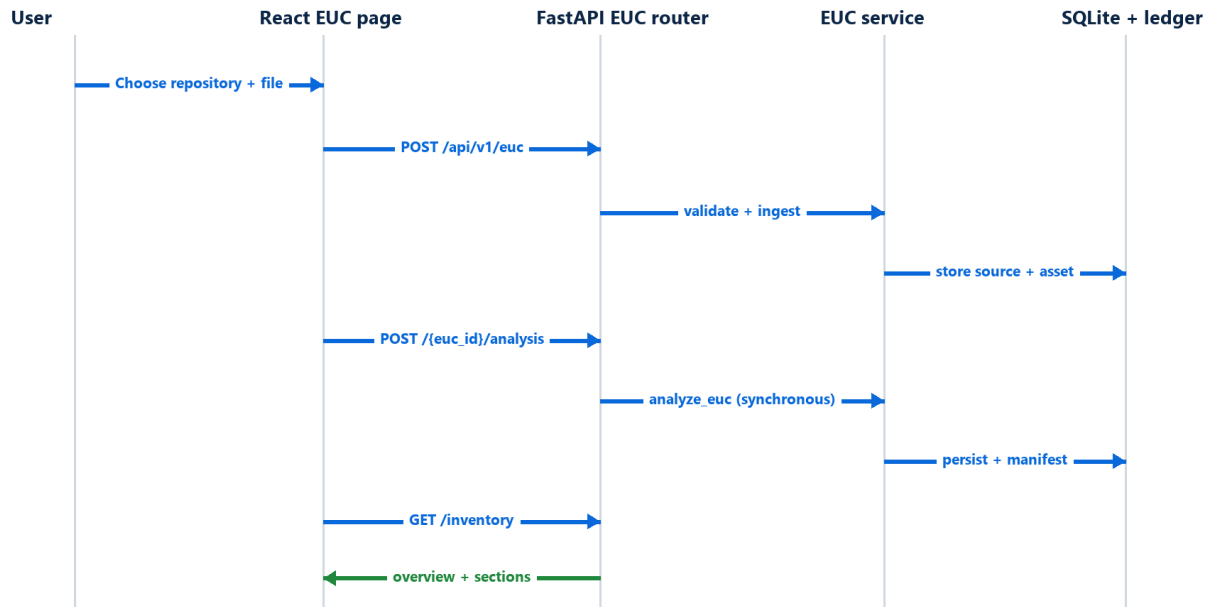


Figure 3. Current user-interface and API sequence.

5. Feature block specifications

5.1 Block A - repository selection and EUC registration

The React page obtains editable repositories from the existing dataset/repository foundation. A user selects a repository and file. The client sends multipart form data containing repository_id and file to POST /api/v1/euc. The backend reads no more than the configured maximum plus one byte, resolves the authenticated principal and passes the request to ingest_euc.

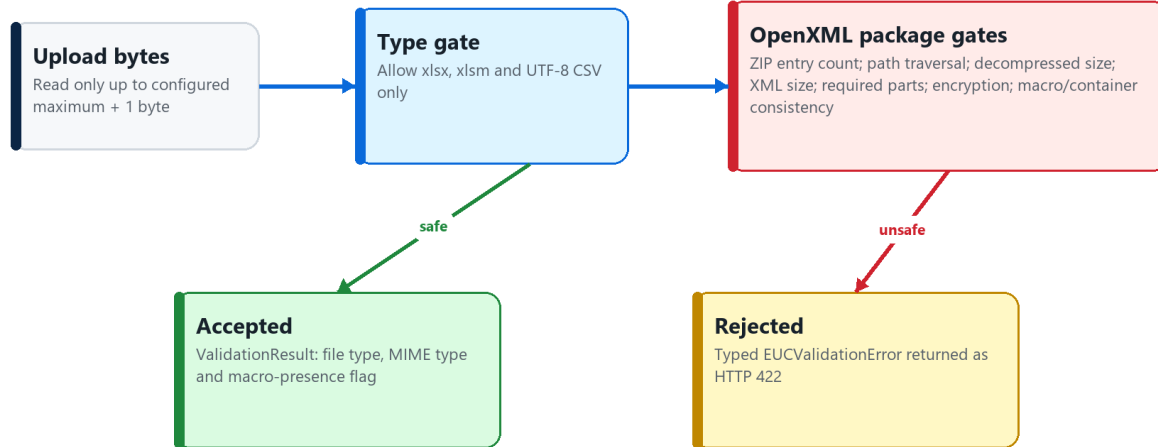
Input	Processing	Output
Authenticated user; repository ID; filename; source bytes	Repository edit permission, validation, hashes, deduplication and ledger storage	EUC asset with PENDING status or existing deduplicated asset

5.2 Block B - hostile-file-aware validation

Validation occurs both during registration and immediately before analysis. CSV files must be non-empty UTF-8 text. OpenXML files must be valid ZIP packages with required workbook parts. Package paths, entry counts, decompressed size, XML part size, encryption and macro/container consistency are checked before parsing.

Validation and hostile-file safety

Low-level validation gates before source registration



Configured defaults: 50 MB source, 500 MB decompressed, 5,000 entries, 100 MB XML, 5M cells, 1M formulas, 300 s analysis.

Figure 4. Validation gates and typed rejection path.

Condition	Error code	Default behavior
Unsupported extension	UNSUPPORTED_FORMAT	Reject anything except xlsx, xlsxm and csv.
Invalid or missing OpenXML parts	CORRUPT_OPENXML	Reject malformed packages.
Unsafe path	PATH_TRAVERSAL	Reject absolute or parent-traversing package entries.
Excessive entries or expansion	ZIP_BOMB	Reject package expansion beyond limits.
Encrypted workbook	PASSWORD_PROTECTED	Reject because structure cannot be safely inventoried.
Macro payload in xlsx	INVALID_FILE	Reject inconsistent container; require xlsxm.

5.3 Block C - fingerprinting, deduplication and immutable source

`content_fingerprint` computes SHA-256 over exact bytes. `structure_fingerprint` computes a canonical package-part signature excluding media and calcChain, or a CSV signature based on the first line. Repository-scoped content deduplication prevents duplicate EUC_ASSETS records for identical bytes. The original is stored in the semantic ledger as EUC_SOURCE_FILE, linked through object references, and retained as a garbage-collection root.

Content identity and immutable storage

Low-level registration, deduplication and provenance

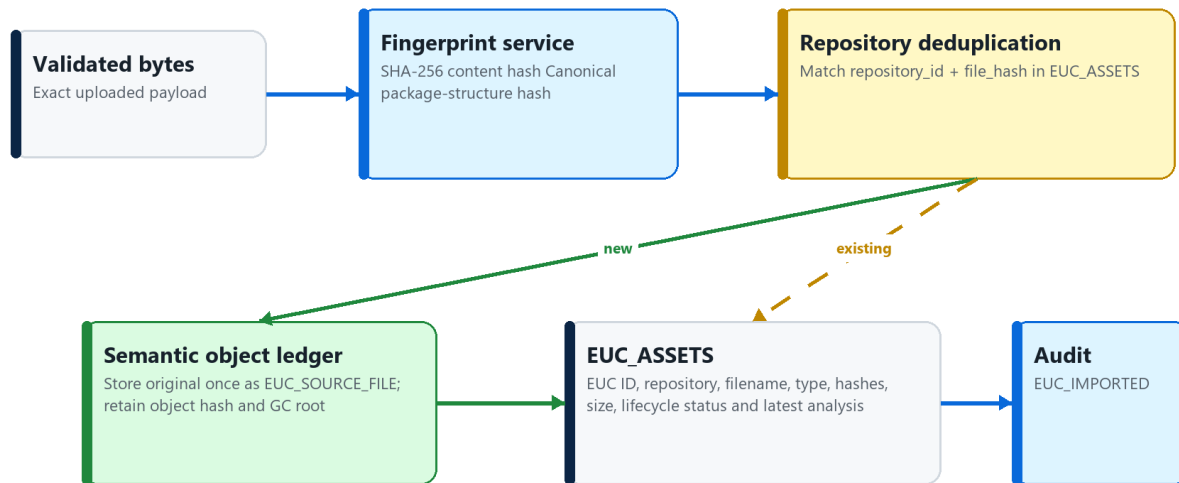


Figure 5. Source identity, object storage and deduplication.

5.4 Block D - analysis orchestration

`analyze_euc` creates a unique analysis run, associates it with the repository default branch head, records status transitions, reloads and revalidates the immutable source, writes a temporary file, and creates `AnalysisContext`. It then invokes the three analyzers, aggregates warnings and metrics, persists normalized records, stores the immutable manifest, updates the asset's latest-analysis pointer, records duration and audit events, and deletes the temporary file.

Static analysis orchestration

Low-level analyzer pipeline with status and timeout controls

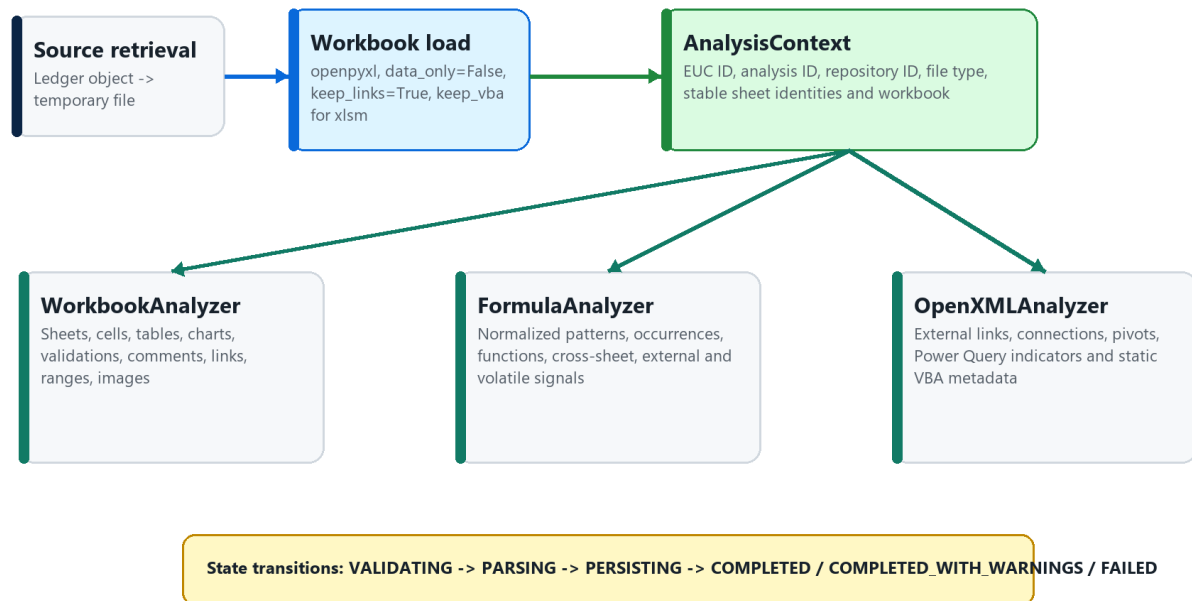


Figure 6. Analysis orchestration and result aggregation.

State	Meaning
PENDING	Source registered; no analysis started.
VALIDATING	Source retrieval and validation in progress.
PARSING	Workbook and package analyzers running.
PERSISTING	Normalized records and manifest being written.
COMPLETED	Analysis completed without warnings.
COMPLETED_WITH_WARNINGS	Usable inventory completed with static-analysis warnings.
FAILED	Analysis failed; errors are recorded against the run and asset.

5.5 Block E - workbook structural analyzer

WorkbookAnalyzer iterates every worksheet and cell through openpyxl. It counts used, formula, constant and styled-blank cells; captures comments and hyperlinks; records sheet visibility and hidden axes; inventories tables, validations, merged ranges, charts and images; and creates workbook-level metrics. Stable sheet IDs are reused from repository semantics where available, otherwise derived deterministically from the sheet name.

Workbook structural discovery

WorkbookAnalyzer low-level extraction model

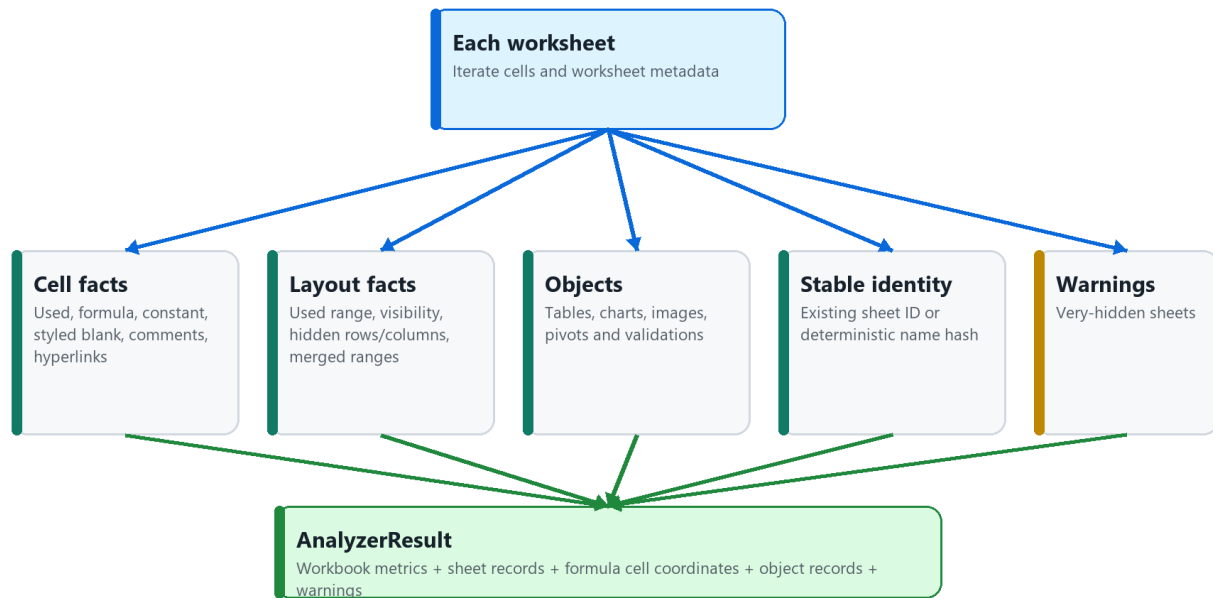


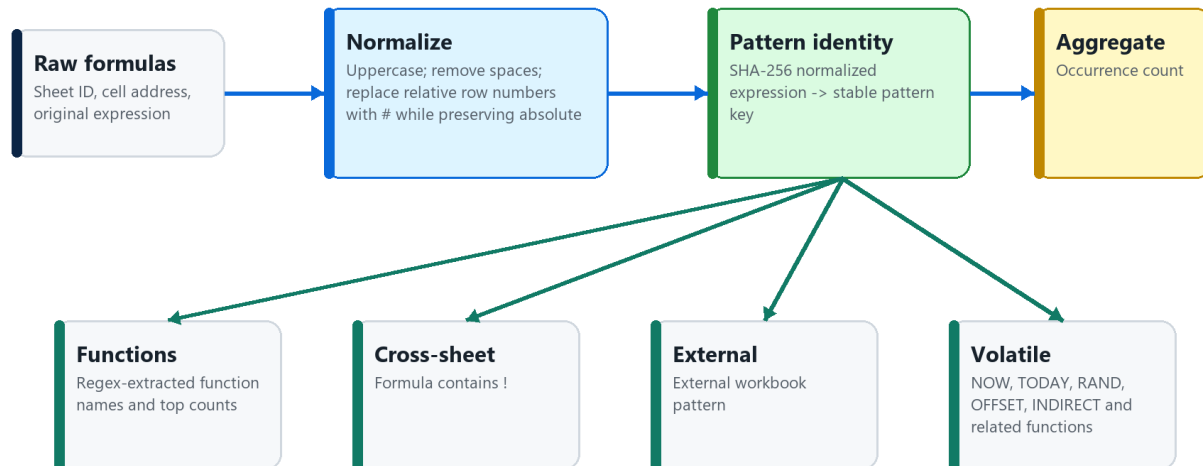
Figure 7. WorkbookAnalyzer extraction model.

5.6 Block F - formula pattern analyzer

FormulaAnalyzer consumes formula-cell records from WorkbookAnalyzer. It normalizes formulas so repeated relative-row logic groups into the same pattern, hashes each normalized expression, records every occurrence, extracts function names, and flags cross-sheet, external-workbook and volatile behavior. This produces a compact semantic view of repeated business logic while retaining raw formulas for traceability.

Formula pattern discovery

FormulaAnalyzer low-level normalization and classification



Purpose: compare repeated business logic semantically instead of reviewing every formula occurrence independently.

Figure 8. Formula normalization, identity and classification.

TECHNICAL BOUNDARY

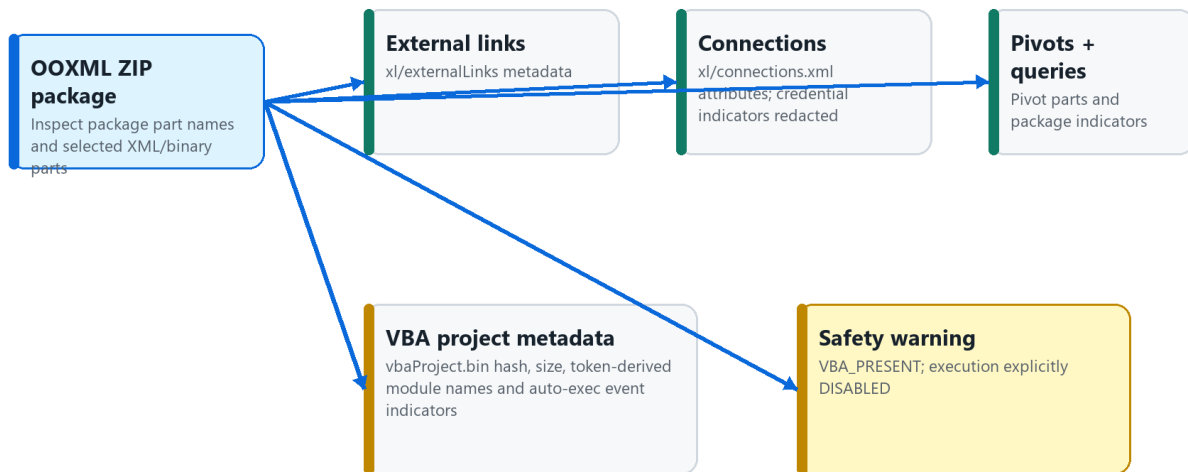
Formula normalization is regex-based and intentionally lightweight. It groups common A1-style patterns but is not a complete Excel grammar or calculation engine.

5.7 Block G - OpenXML, external connection and VBA metadata analyzer

OpenXMLAnalyzer opens the validated workbook as a ZIP package. It detects external-link parts, packaged connection attributes, pivot definitions, Power Query indicators and vbaProject.bin. Credential-like connection attributes are redacted. VBA discovery records binary size and hash plus token-derived candidate module names and auto-execution event names. It does not execute VBA or fully decompile procedures.

OpenXML, connection and VBA metadata discovery

Static package inspection; no macro or connection execution



Boundary: this is metadata discovery, not full VBA parsing, query execution, credential validation or behavioral analysis.

Figure 9. Static OpenXML and VBA metadata inspection.

5.8 Block H - normalized persistence and manifest

The service persists each analyzer result into purpose-specific tables. Formula occurrences are inserted in batches of 2,000. A summary merges workbook, formula and package metrics. The manifest records analyzer version, EUC ID, source commit, structure hash, summary, sheets, formula patterns, external links, connections, objects and warnings. The manifest is stored in the semantic ledger and references the immutable source object.

Normalized inventory and immutable manifest

Low-level persistence model

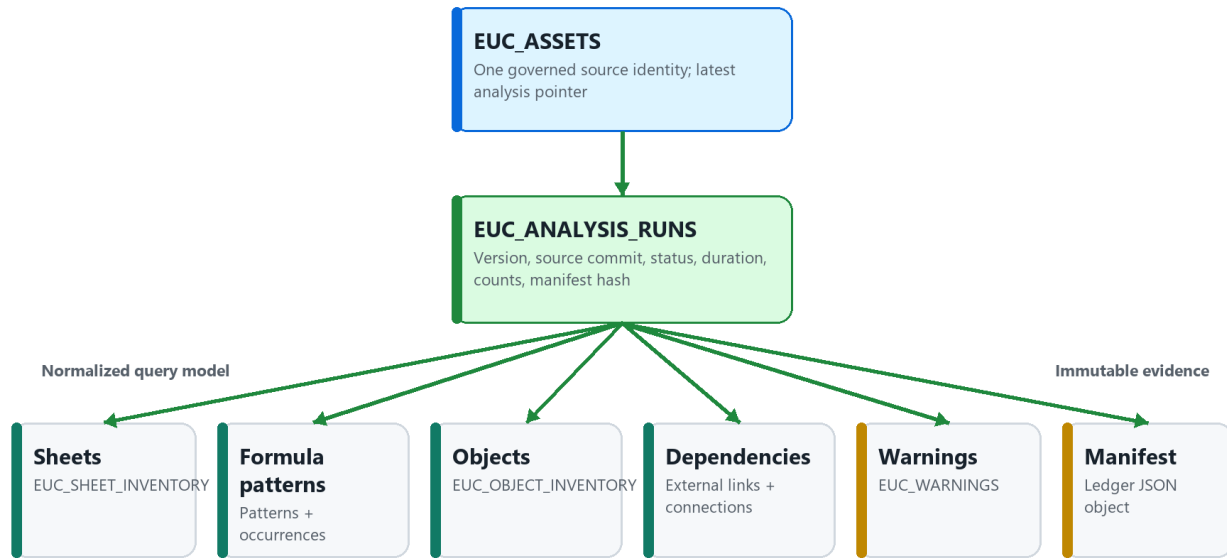


Figure 10. Normalized query model and immutable evidence model.

5.9 Portfolio, inventory views and export

GET /api/v1/euc lists accessible assets with repository metadata, latest status, analyzer version, warnings and summary. GET /inventory supports overview, sheets, formulas, objects and dependencies sections with limit/cursor pagination. The React page requests up to 250 records per section, renders summaries and lists, and exposes re-analysis and JSON export.

View	Primary evidence shown
Registered assets	Filename, repository, size, sheets, formulas and analysis status.
Workbook map	Sheet visibility, ranges, cell counts, formulas, tables, charts, validations and hidden axes.
Formula patterns	Normalized expression, functions, occurrence count and behavioral signals.
Objects	Grouped tables, named ranges, comments, hyperlinks, validations, charts, images, VBA and query indicators.
Dependency sources	External workbook references and packaged data connections.
Export JSON	The latest immutable manifest payload.

6. Data model and persistence

6.1 Core entities

Table	Key relationship	Purpose
EUC_ASSETS	EUC_ID; repository ID; latest analysis ID	Persistent source identity and lifecycle.
EUC_ANALYSIS_RUNS	ANALYSIS_ID -> EUC_ID	Versioned run status, provenance, metrics, manifest and errors.
EUC_SHEET_INVENTORY	ANALYSIS_ID + SHEET_ID	Worksheet structure and counts.
EUC_FORMULA_PATTERNS	ANALYSIS_ID + normalized hash	Grouped formula logic and classification.
EUC_FORMULA_OCCURRENCES	ANALYSIS_ID + pattern + cell	Raw formula evidence at cell level.
EUC_OBJECT_INVENTORY	ANALYSIS_ID + optional sheet	Workbook and worksheet object metadata.
EUC_EXTERNAL_LINKS	ANALYSIS_ID	External workbook relationships.
EUC_CONNECTIONS	ANALYSIS_ID	Packaged data-connection metadata.
EUC_WARNINGS	ANALYSIS_ID	Non-fatal analysis warnings.

6.2 Provenance chain

The provenance chain is: repository -> EUC asset -> immutable source object -> analysis run -> normalized records -> immutable manifest. The analysis run also records the repository source commit and analyzer version. The manifest object references the original source object, ensuring both remain storage roots.

WHY TWO STORAGE MODELS

Normalized SQL supports filtering, ordering and UI pagination. Immutable ledger objects support stable evidence, content addressing, audit and reproducibility. Both are required for an operational inventory platform.

7. API and user-interface contracts

7.1 Stage 2.1 endpoints

Method and path	Permission / input	Result
POST /api/v1/euc	Authenticated principal; euc.analyze; multipart repository_id + file	201 asset record; may be deduplicated.
GET /api/v1/euc	Accessible repositories; optional repository_id and search	Portfolio assets with latest summary.
POST /api/v1/euc/{euc_id}/analysis	euc.analyze	Synchronous completed or failed analysis response.
GET /api/v1/euc/{euc_id}/analysis/{analysis_id}	euc.read	Status, progress, freshness, summary, warnings and errors.
GET /api/v1/euc/{euc_id}/inventory	euc.read; include, limit and cursor	Selected normalized inventory sections.
GET /api/v1/euc/{euc_id}/export	euc.read; completed analysis required	JSON manifest attachment.

7.2 Error mapping

The router converts repository permission failures to HTTP 403, missing assets or analyses to 404, EUC validation failures to 422 with a typed code, invalid requests and timeouts to 400, and unexpected analysis failures to 500. This gives the client a stable distinction between user-correctable input and server failure.

7.3 Pagination and response behavior

The inventory API applies the same limit and cursor to each requested section. `next_cursor` is returned if any selected list reaches the limit. The current UI requests 250 records per section once and does not follow `next_cursor`. Large inventories can therefore be partially displayed even though additional records remain available through the API.

8. EUC Inventory and AI Command Center

8.1 Architectural responsibility split

Deterministic EUC layer	AI layer
Counts sheets, cells, formulas, objects and connections	Explains why those facts matter.
Builds formula and dependency structures	Summarizes impact and recommends review.

Deterministic EUC layer	AI layer
Generates rules, scores and findings	Communicates findings in user language.
Stores source and manifest hashes	Cites evidence IDs and provenance.
Enforces deterministic control states	Must not replace PASS/FAIL authority.

CORE AI PRINCIPLE

The parser determines the facts. Graph and control engines determine relationships and deterministic findings. The LLM explains, compares and recommends using those authorized facts.

8.2 Implemented connection

The current EvidenceRetriever enforces ai.use and repository.read, adds deterministic repository KPIs and relevant audit events, then queries open EUC_FINDINGS for assets in the selected repository. Each evidence item carries type, ID, title, summary, classification, freshness and a source hash. Evidence text is explicitly labeled untrusted evidence rather than instructions, and injection-like content is isolated with warnings.

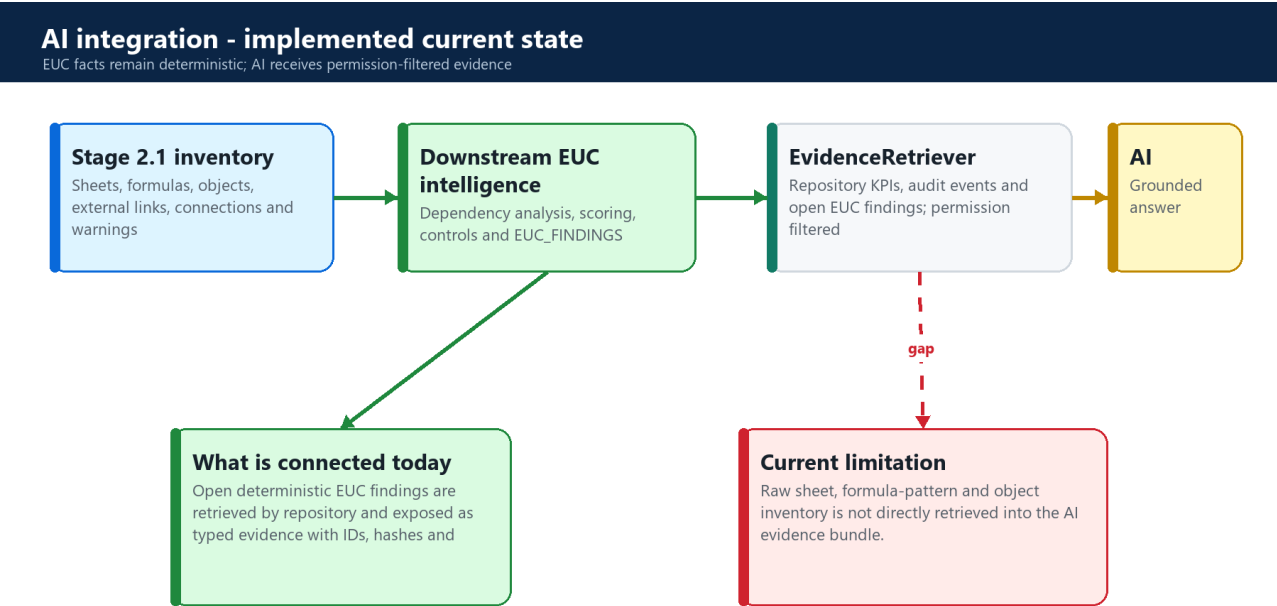


Figure 11. Implemented EUC-to-AI evidence path and present raw-inventory gap.

8.3 AI gateway controls

- Organization and feature policies determine whether AI and external providers are allowed for the evidence classification.
- Model routing selects an enabled governed model and active prompt version.
- Organization and per-user token quotas are enforced before provider execution.
- Evidence context receives a stable context hash and request-to-evidence references are persisted.
- Model output must validate against the GroundedAnswer schema; malformed output is repaired up to three attempts.
- Citations not present in the retrieved evidence allow-list are removed.
- A grounding score combines retrieval quantity and citation coverage; low evidence sets insufficient_evidence.
- Responses, conversations, token usage, latency and audit events are persisted.

8.4 Recommended direct EUC evidence adapter

To answer detailed questions such as 'Which sheets contain volatile formulas?' or 'Summarize the external connections in this workbook,' the AI layer should retrieve Stage 2.1 inventory sections directly rather than relying only on downstream findings. The adapter should accept repository and optional EUC scope, invoke existing authorized service methods or read-only tools, limit and summarize large occurrence sets deterministically, and emit typed evidence records with analysis ID, manifest hash, source commit and freshness.

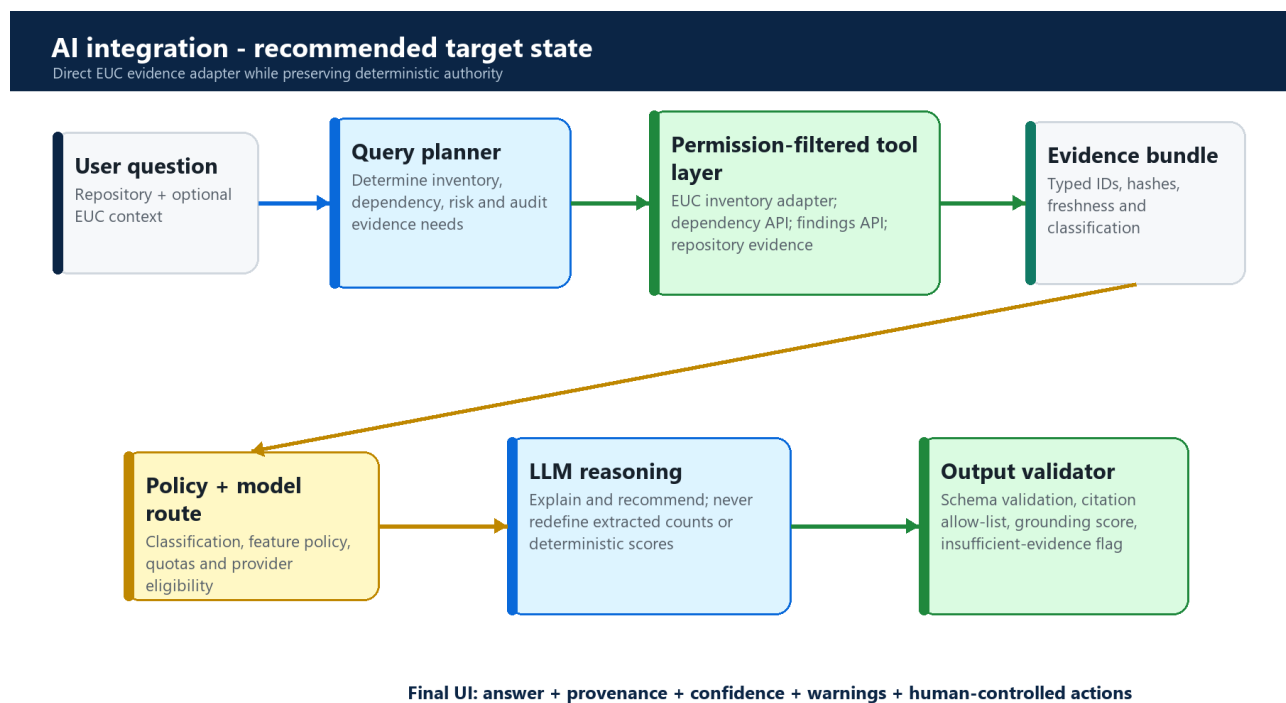


Figure 12. Recommended direct inventory grounding architecture.

Recommended evidence type	Source	Use
---------------------------	--------	-----

Recommended evidence type	Source	Use
EUC_OVERVIEW	Latest analysis summary	Workbook-scale explanation and high-level risk context.
EUC_SHEET	EUC_SHEET_INVENTORY	Sheet map, visibility and structural questions.
EUC_FORMULA_PATTERN	EUC_FORMULA_PATTERNS	Business-logic and volatile/external formula questions.
EUC_OBJECT_SUMMARY	EUC_OBJECT_INVENTORY	Tables, named ranges, VBA, Power Query and validation questions.
EUC_CONNECTION	EUC_CONNECTIONS / EUC_EXTERNAL_LINKS	External system and workbook dependency explanation.
EUC_MANIFEST	Immutable manifest hash	Provenance and reproducibility reference.

9. Security, governance and non-functional behavior

Security, governance and trust boundaries

Controls applied across EUC Inventory and AI

Identity and authorization

Authenticated principal; repository read/edit; ai.use / ai.read; organization and classification policy

Hostile input isolation

ZIP safety gates; password rejection; macro non-execution; credential attribute redaction; temporary-file cleanup

Provenance and immutability

Content hashes; structure hashes; source object; analyzer version; manifest hash; audit events

AI grounding controls

Evidence-is-not-instructions marker; injection detection; allowed citation filter; structured output; quotas

Figure 13. Security and trust controls spanning EUC Inventory and AI.

9.1 Access control

Repository access is resolved through active repository ownership or membership, followed by `authorization_engine` checks for `euc.analyze` or `euc.read`. AI applies organization-level `ai.use` or `ai.read` plus `repository.read`. The AI gateway additionally enforces classification and feature model policy.

9.2 Non-execution safety

`openpyxl` loads formulas without calculating them because `data_only=False`. `keep_vba` preserves VBA binary content for metadata inspection when the file is `xlsm`. The code does not launch Excel, invoke VBA, refresh connections or submit workbook content to an execution runtime. Temporary source files are removed in a finally block.

9.3 Limits and defaults

Control	Default
Maximum source file	50 MB
Maximum decompressed package	500 MB
Maximum ZIP entries	5,000
Maximum XML part	100 MB
Maximum cells	5,000,000
Maximum formulas	1,000,000
Analysis timeout	300 seconds
Inventory API limit	1-500 rows per requested section

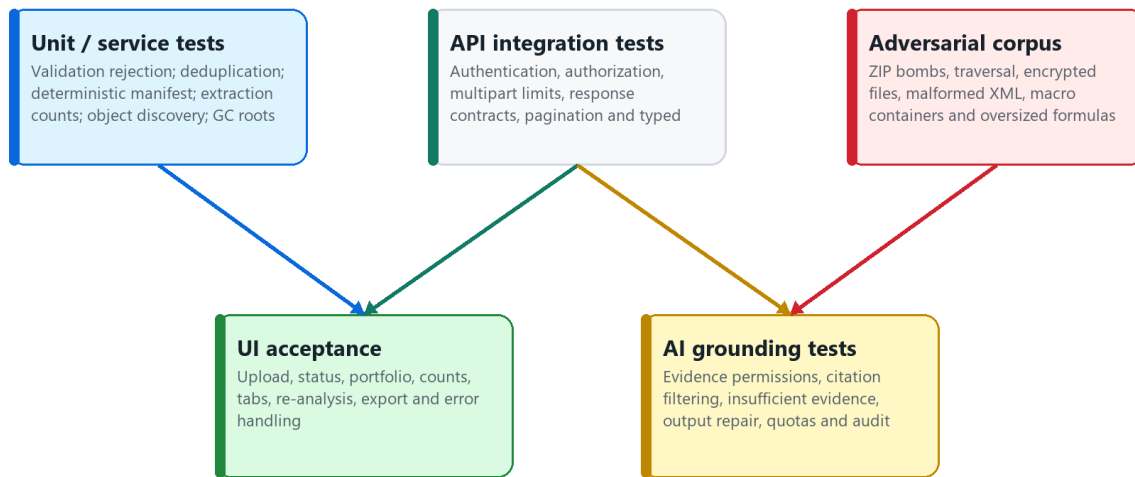
9.4 Observability and audit

Registration, analysis start, completion and failure generate audit events. Completed analysis records duration metrics and result status. AI requests record model, prompt, context references, latency, token usage, output validity, grounding confidence and cache behavior. These records enable technical support, governance review and cost control.

10. Testing and acceptance strategy

Verification strategy

Automated coverage, manual acceptance and production readiness



Release gate: deterministic tests pass, representative workbooks reconcile, and every document/UI artifact is visually reviewed

Figure 14. Recommended layered verification strategy.

10.1 Existing automated tests

backend/tests/test_stage21_euc_inventory.py creates a temporary product database and a representative workbook containing formulas, a very-hidden sheet, a table, data validation and a named range. It verifies invalid input rejection, repository-scoped deduplication, deterministic manifest hashes across re-analysis, expected formula counts, extracted sheets and objects, JSON export and storage garbage-collection roots.

VERIFIED BASELINE

The dedicated Stage 2.1 suite was executed during this documentation task: 3 tests passed. The frontend production build also completed successfully, with only webpack bundle-size warnings.

10.2 How to run the automated tests

Backend Stage 2.1:

```
cd C:\Projects\board-walk-clone-1\backend
python -m pytest tests\test_stage21_euc_inventory.py -q
```

Frontend production build:

```
cd C:\Projects\board-walk-clone-1\frontend
npm.cmd run build
```

10.3 Manual acceptance procedure

1. Start the backend on <http://localhost:8000> and the frontend on <https://localhost:3000>.
2. Authenticate, create or select an editable repository, and open EUC Inventory.
3. Upload a representative xlsx containing formulas, a table, validation, named ranges and a hidden sheet.
4. Confirm source registration, completed status and workbook-map counts against Excel inspection.
5. Confirm formula-pattern grouping and the cross-sheet, external and volatile indicators.
6. Upload a representative xlsx and verify VBA_PRESENT while confirming that no macro side effect occurs.
7. Export the JSON manifest and reconcile its analysis version, summary, objects and warnings.
8. Upload the identical bytes again and verify the same EUC ID is returned as deduplicated.
9. Modify one formula, upload again and verify a new content hash plus changed formula evidence.
10. Validate a user without repository access receives 403 and an invalid package receives typed 422 detail.

10.4 Recommended missing tests

- Direct API tests for multipart byte limits, authentication, repository authorization and every error mapping.
- Dedicated CSV behavior, UTF-8 rejection, empty header and maximum-cell tests.
- Adversarial OpenXML fixtures for path traversal, high compression ratio, excessive entries, oversized XML and encryption.
- Real xlsx fixtures validating macro-container handling, warning content and no-execution guarantees.
- Inventory pagination tests where multiple sections independently cross the same cursor boundary.
- Failure-injection tests proving transaction rollback and temporary-file cleanup.
- Direct EUC-to-AI retrieval tests once the recommended raw-inventory adapter is implemented.
- UI tests for asynchronous error states, re-analysis and large portfolio/search behavior.

11. Advantages, limitations and recommended improvements

11.1 Advantages

- Transforms opaque workbook binaries into queryable technical evidence.
- Creates a safer discovery process by avoiding VBA and connection execution.
- Reduces duplicate storage and analysis ambiguity through cryptographic identity.
- Supports repeatable analysis and audit through versioned manifest hashes.
- Compresses repeated spreadsheet logic into formula patterns while retaining raw occurrences.
- Provides a reusable foundation for dependency, risk, control and migration features.
- Enables AI explanation over governed findings instead of allowing an LLM to guess workbook structure.

11.2 Current limitations

Limitation	Engineering impact
Only xlsx, xlsx and UTF-8 csv	Legacy xls and xlsm require additional parsers or conversion.

Limitation	Engineering impact
Synchronous analysis endpoint	Large files can hold requests for up to the configured timeout.
Regex formula normalization	Not a complete Excel grammar; structured references and dynamic constructs may be imperfectly grouped.
VBA token discovery	Does not produce a complete module/procedure/call graph.
Power Query package indicator	Does not fully parse M expressions or execute queries.
UI requests first 250 rows	Large sections require pagination support in the client.
AI retrieves findings, not raw inventory	Detailed workbook questions lack direct Stage 2.1 grounding today.
Structure hash uses names and sizes	It is useful for comparison but is not a semantic workbook-equivalence proof.

11.3 Prioritized improvements

1. Add a direct, permission-filtered EUC evidence adapter to the AI retrieval layer.
2. Move analysis to a background worker and expose polling or server-sent progress while retaining analysis-run state.
3. Implement frontend cursor pagination and section-specific limits.
4. Adopt a full Excel formula parser for richer normalization and structured-reference handling.
5. Add deeper VBA and Power Query parsers in isolated static-analysis components.
6. Build an adversarial workbook corpus and performance baseline for configured limits.
7. Expose manifest/source freshness and repository-head staleness directly in the EUC UI.

12. Codebase traceability and dependencies

12.1 Primary implementation files

Path	Responsibility
frontend/src/App.jsx	EUC state, lifecycle calls and inventory rendering.
frontend/src/services/api.js	EUC REST client functions.
frontend/src/styles.css	EUC page visual system and responsive layout.
backend/app/api/euc.py	Authenticated API boundary and error mapping.

Path	Responsibility
backend/app/euc/service.py	Registration, orchestration, persistence, retrieval and export.
backend/app/euc/validation.py	Hostile-file-aware source validation.
backend/app/euc/fingerprint.py	Content and structure fingerprints.
backend/app/euc/analyzers.py	Workbook, formula and OpenXML analyzers.
backend/app/euc/models.py	Shared analyzer contracts.
backend/app/database.py	Core EUC and downstream database schema.
backend/app/ai/retrieval.py	Permission-filtered evidence retrieval including EUC findings.
backend/app/ai/gateway.py	AI policy, routing, quota, structured output and citation controls.
backend/app/ai/service.py	Conversation, evidence and AI request orchestration.
backend/tests/test_stage21_euc_inventory.py	Dedicated Stage 2.1 automated tests.
backend/tests/test_stage5_ai_platform.py	AI grounding, citation filtering, quota and agent audit tests.

12.2 Direct technology dependencies

Dependency	Use in this feature
Python 3	Backend services, static analysis and persistence orchestration.
FastAPI	Authenticated upload, inventory and export HTTP endpoints.
Pydantic	AI structured output and request validation; API request models.
openpyxl	Workbook structure and formula access without Excel execution.
python-multipart	Multipart file upload support.
SQLite	Normalized inventory and operational metadata.
React 18	EUC Inventory and AI Command Center user interfaces.
Web APIs	FormData, authenticated fetch and JSON download.

Dependency	Use in this feature
Python standard library	ZIP/XML/CSV parsing, hashing, JSON, temporary files and timing.

Appendix A - Operational runbook

A.1 Local startup

1. Install backend requirements from backend/requirements.txt.
2. From backend, start uvicorn app.main:app --reload --port 8000.
3. From frontend, run npm install if dependencies are absent, then npm.cmd start.
4. Open <https://localhost:3000> and accept the local development certificate if required.
5. Use <http://localhost:8000/docs> for authenticated API contract inspection.

A.2 Failure triage

Symptom	Check
Upload rejected	HTTP detail code, extension, size, ZIP structure, encryption and repository edit permission.
Analysis failed	EUC_ANALYSIS_RUNS errors_json, asset status, audit event and configured limits.
Counts differ from Excel	Merged/styled blanks, formula data type, hidden axes, unsupported objects and analyzer version.
Inventory appears incomplete	API include list, limit/cursor and frontend 250-row request limit.
AI answer lacks workbook detail	Current retriever exposes EUC findings, not raw inventory; inspect evidence_bundle.
AI citation removed	Citation ID/type was not in the authorized retrieved evidence allow-list.

Appendix B - Acceptance checklist

- Supported files register only within authorized repositories.
- Unsafe, encrypted and malformed sources are rejected with typed errors.
- Identical source bytes deduplicate within the repository.
- Original source and manifest remain immutable storage roots.
- No VBA, formula calculation or external connection execution occurs.
- Sheet, formula, object, link and connection evidence reconciles with representative workbooks.
- Re-analysis produces deterministic manifest identity for unchanged inputs and analyzer version.

- Inventory export includes source and analysis provenance.
- AI responses cite only authorized evidence and signal insufficient grounding.
- Target-state direct raw-inventory AI retrieval is not claimed until implemented and tested.

Appendix C - Glossary

Term	Definition
AIR	Application Intermediate Representation used by the downstream native-model feature.
EUC	End-User Computing asset, typically a spreadsheet used to implement business logic or operations.
GC root	An object-store reference that prevents immutable source or manifest evidence from garbage collection.
Grounding	Constraining AI output to retrieved evidence and validating cited evidence references.
OOXML	Office Open XML package format used by xlsx and xlsxm workbooks.
Semantic inventory	Normalized technical representation of workbook structures and logic.
Static analysis	Inspection without executing workbook formulas, macros, queries or external connections.
Volatile formula	A formula using functions that may recalculate frequently or create difficult dependency behavior.

Conclusion

EUC Inventory is not merely a file-upload page. It is the deterministic discovery and provenance layer that converts a governed workbook source into technical evidence. Its design separates extraction from interpretation: parsers and graph/control engines establish facts, while AI provides authorized explanation and recommendations. The current code already grounds AI in repository evidence and open EUC findings. Adding a direct Stage 2.1 evidence adapter is the clearest next step for detailed workbook-aware AI without weakening the deterministic source of truth.